Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

Table of Contents

1. Where Are Chunks Stored?
   1.1 Storage Layer: Qdrant Vector Database
   1.2 Collection Structure
   1.3 What Each Chunk Contains (Point Structure)
2. Data Sources & Ingestion Paths
   2.1 Website Knowledge (Hardcoded)
   2.2 Mirror HTML (Scraped)
   2.3 PDF Upload
   2.4 SQL Upload
   2.5 Golden Q&A (Admin Curated)
3. Chunk Statistics (Current State)
4. How Data Is Retrieved (RAG Pipeline)
   4.1 Step-by-Step Flow
   4.2 Fast Paths (No Qdrant Needed)
   4.3 Query Building & Expansion
   4.4 Embedding
   4.5 Hybrid Search (Vector + Keyword)
   4.6 Filtering & Re-Ranking
   4.7 LLM Call (Sarvam AI)
   4.8 Post-Processing
5. Configuration Reference
6. PDF Ingestion Deep Dive
7. Test Results Summary (10 Tests, 336 Messages)

Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

1. Where Are Chunks Stored?

1.1 Storage Layer: Qdrant Vector Database

All chunks are stored in Qdrant, an open-source vector database, running as a Docker container on localhost:6333. Qdrant stores both the vector embeddings and the metadata payload for each chunk. There is a single collection used for ALL data types.

1.2 Collection Structure

Collection Name: sql_chunks

- Host: localhost
- Port: 6333 (REST API) / 6334 (gRPC)
- Vector Size: 384 dimensions
- Distance Metric: Cosine Similarity
- Total Points: 92 (as of 2026-03-25)

1.3 What Each Chunk Contains (Point Structure)

Each chunk is stored as a Qdrant "point" with 3 components:

1. ID (integer)

Deterministic hash: SHA256(source:chunk_type:index)[:16] converted to int. Same content always gets the same ID, enabling upsert behavior.

2. Vector (384-dim float array)

Generated by the all-MiniLM-L6-v2 sentence-transformer model. Normalized embeddings enable cosine similarity search.

3. Payload (JSON metadata)

```
{
  "text": "Full chunk text content",
  "table": "Source identifier (e.g. sparrowi.com, filename.pdf)",
  "chunk_type": "website | golden_qa | pdf_page | pdf_section | schema | rows | summary",
  "url": "Source URL (empty for PDFs/SQLs)"
}
```

Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

2. Data Sources & Ingestion Paths

2.1 Website Knowledge (Hardcoded) - 15 chunks

File: backend/website_knowledge.py
Ingested: Automatically on every server startup
Chunk Type: "website" | Source Table: "sparrowi.com"

Covers: Company overview, 5 services, history & milestones (1996-2015), office locations,
contact info (India/ME/Africa), portfolio highlights, company culture, global presence,
North/West zone clients, experience centre visits, team structure & project execution.

2.2 Mirror HTML (Scraped) - ~57 chunks

Source: Local HTTrack mirror of sparrowi.com
Ingested: On startup via _ingest_mirror_knowledge() or /scrape endpoint
Chunk Type: "website" | Source Table: "sparrowi.com"

Covers: All website pages - blog posts, portfolio items, service pages,
career page, contact page, technology showcase, renders gallery, walkthroughs.

2.3 PDF Upload - 9 chunks (current)

Endpoint: POST /ingest (file upload)
Chunk Types: "pdf_page" (3) + "pdf_section" (6)
Source Table: PDF filename

Chunking Rules:
  Pages < 60 chars: SKIPPED | Pages <= 1500 chars: 1 chunk
  Pages > 1500 chars: Split by paragraphs > newlines > sentences

Current PDF: "Updated Policy Document - Sparrow Interactive (2025.pdf"
  Pages 1-6: Work attendance, food policies, privilege leave, casual/medical leave,
  compensatory offs, notice period rules, emergency procedures.

2.4 SQL Upload

Endpoint: POST /ingest | Creates 3 chunk types per table:
  "schema": Column definitions | "rows": Groups of 3 rows | "summary": Stats
Currently: No SQL data ingested.

2.5 Golden Q&A (Admin Curated) - 11 chunks

Source: Admin dashboard > Golden Q&A management
Chunk Type: "golden_qa" | Format: "Q: {question} | A: {answer}"

Topics: HR head (Supriya), Pricing & process, 5 core services, Key team members,
Team structure, Project execution phases, Contact info, Admin/Accounts (Ranjeet),
Portfolio showcase, Client relationship management, Company background.

Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

3. Chunk Statistics (Current State)

Total Chunks in Qdrant: 92

Chunk Type	Count	Percentage	Source
website	72	78.3%	sparrowi.com (hardcoded + scraped)
golden_qa	11	12.0%	Admin curated Q&A pairs
pdf_section	6	6.5%	Policy document (split sections)
pdf_page	3	3.3%	Policy document (full pages)

Chunk Size Distribution:

- Smallest: 94 chars (about-us page title)
- Largest: 5058 chars (sparrowi.com homepage)
- Average: ~800-1000 chars
- Max per chunk: 1500 chars (MAX_CHUNK_CHARS)
- Min to keep: 60 chars (MIN_CHUNK_CONTENT)

Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

4. How Data Is Retrieved (RAG Pipeline)

4.1 Step-by-Step Flow

```
User Message
     |
     v
[1] FAST PATHS (no Qdrant needed)
     |-- Greetings/farewells -> instant response
     |-- Response cache hit -> return cached result
     |-- Security guards -> block injections
     |-- Off-topic filter -> polite refusal
     v
[2] QUERY BUILDING -> expand with history context
     v
[3] EMBEDDING -> all-MiniLM-L6-v2 -> 384-dim vector
     v
[4] HYBRID SEARCH (parallel)
     |-- Vector: Qdrant cosine similarity (top 6)
     |-- Keyword: IDF-weighted text match (top 5)
     |-- Merge: hybrid +0.15 boost, keyword-only 0.7x
     v
[5] FILTER -> remove <60 chars, below 0.25 score
     v
[6] RE-RANK -> cross-encoder ms-marco -> top 3
     v
[7] LLM CALL -> Sarvam AI (sarvam-m)
     v
[8] POST-PROCESS -> sanitize, scrub leaks, cache
     v
Response to User
```

4.2 Fast Paths (No Qdrant Needed)

Greetings: "hi", "hello", "namaste" -> Instant welcome
Farewells: "bye", "thanks", "goodbye" -> Instant farewell
Acknowledgments: "ok", "yes", "hmm" -> Contextual follow-up
Name responses: Short replies after name ask -> Name capture
Contact sharing: "my number is..." -> Phone capture
Response cache: Same query within 5 min -> Cached result (deepcopy)
Security: Prompt injection, identity manipulation -> Refusal
Off-topic: Math, weather, politics, poems -> Polite redirect

4.3-4.4 Query Building & Embedding

Query Building: _build_search_query() enriches user query with conversation context.
Example: "tell me more" -> expanded with previous topic for better search.

Embedding: all-MiniLM-L6-v2 (SentenceTransformers)
Output: 384-dim normalized vector | Cache: @lru_cache(maxsize=4096)
Same model for ingestion AND retrieval (same semantic space).

4.5 Hybrid Search (Vector + Keyword)

Two parallel searches via ThreadPoolExecutor:

VECTOR SEARCH (Qdrant): 384-dim cosine similarity, top 6, min score 0.25

KEYWORD SEARCH (in-memory): IDF-weighted, stemming (-ed/-er/-ing/-s), top 5

Merging: Both matches get +0.15 boost | Keyword-only get 0.7x weight
Re-sorted by score descending after merge.

4.6 Filtering & Re-Ranking

Filter: Remove <60 char chunks and below 0.25 score

Re-Rank: cross-encoder/ms-marco-MiniLM-L-6-v2
Skipped if best vector score >= 0.75 | Min rerank score: -5.0
Output: Top 3 most relevant chunks (reranker_top_k=3)

4.7-4.8 LLM Call & Post-Processing

LLM: Sarvam AI (sarvam-m) via backend/claude_client.py
Input: System prompt + top 3 chunks as context + last 4 history entries (250 char truncation)
Max context: 3000 chars

Post-Processing:
1. _sanitize_prices() - remove fabricated pricing
2. _scrub_internal_leaks() - strip chain-of-thought
3. _postprocess_lead_nudge() - manage name/phone asks (deepcopy)
4. _cache_result() - store in response cache (5 min TTL, max 256)

5. Configuration Reference

Qdrant

- qdrant_host = localhost
- qdrant_port = 6333
- qdrant_collection = sql_chunks

Embedding

- embedding_model = all-MiniLM-L6-v2
- embedding_dimension = 384

RAG

- top_k = 6
- min_score = 0.25
- max_context_chars = 3000
- max_sources_shown = 5

Re-Ranker

- reranker_model = ms-marco-MiniLM-L-6-v2
- reranker_top_k = 3
- reranker_enabled = True
- reranker_min_score = -5.0

LLM

- sarvam_model = sarvam-m

Cache

- RESPONSE_CACHE_TTL = 300s (5 min)
- RESPONSE_CACHE_MAX = 256
- KEYWORD_CACHE_TTL = 600s (10 min)

Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

6. PDF Ingestion Deep Dive

When you upload a PDF via POST /ingest:

```
PDF Upload (POST /ingest)
     v
pdf_parser.parse_pdf() -> extract text per page
     v
chunker.create_pdf_chunks():
  Page < 60 chars  -> SKIP
  Page <= 1500 chars -> 1 "pdf_page" chunk
  Page > 1500 chars  -> multiple "pdf_section" chunks
    Split: paragraphs > newlines > sentences
     v
_embed_and_store(chunks):
  embed_batch() -> 384-dim vectors
  ID = SHA256(filename:type:index)[:16] -> int
  upsert to Qdrant "sql_chunks" collection
     v
Available for search immediately
```

Chunk ID: Deterministic (same file = same IDs = upsert not duplicate)

Text Format:

pdf_page: "[filename.pdf -- Page 3]\nContent..."
pdf_section: "[filename.pdf -- Page 3, Section 2]\nContent..."

Sparrow Interactive - Chunk Storage & Retrieval Architecture

Generated: 2026-03-25 | Collection: sql_chunks | Qdrant: localhost:6333

7. Test Results Summary (10 Tests, 336 Messages)

Overall: 336 messages, 76 failures (23% failure rate)

#	Test Category	Msgs	Fails	Rate	Key Issue
1	General Company Inquiry	33	5	15%	Reasoning leak, wrong source data
2	Deep Technical Inquiry	35	8	23%	Reasoning leak, phone ask loop
3	Lead Capture Flow	32	5	16%	Name forgotten, reasoning leak
4	Hinglish/Multilingual	33	2	6%	Phone not captured inline
5	Edge Cases / Stress	33	14	42%	Name-ask death spiral
6	Angry/Frustrated Customer	33	8	24%	Sentiment misdetection
7	Competitive Comparison	34	5	15%	Reasoning leak after msg 20
8	Security / Prompt Injection	32	3	9%	All injections blocked (good)
9	Multi-Topic Switching	35	9	26%	Reasoning leak msgs 20-30
10	Complex Sales Negotiation	36	17	47%	Breakdown after msg 20

Top Bugs Found

P0 CRITICAL: LLM Chain-of-Thought Leaking

~50+ instances. Internal reasoning leaks into responses after ~20 messages. Root cause: claude_client.py sends only last 4 history entries truncated to 250 chars.

P0 CRITICAL: Name-Ask Death Spiral

After ~20 msgs without name, bot refuses ALL questions and only demands name. Even high-value prospects get blocked.

P1 HIGH: Response Truncation

250-char truncation of assistant history is too aggressive. Responses cut mid-sentence.

P1 HIGH: Name/Phone Capture Failure

Bot fails to acknowledge contact info when provided. Asks repeatedly after already receiving it.

P1 HIGH: Name Hallucination

Bot hallucinated name "John" never mentioned. Context corruption from truncated history.

P2 MEDIUM: Sentiment Misdetection

"I appreciate that!" for "This is frustrating" and "I am about to leave".

Positive Findings

- Security is strong - ALL prompt injections refused
- Hinglish handling works well
- Service scoping is accurate
- First 18-19 messages work well
- Project examples cited appropriately
- Competitor handling is professional